

Forward and Inverse Kinematics of the Dobot Magician Lite Using MATLAB Rigid-Body Dynamics and URDF-Based Modeling

Jatin Satyam

School for Engineering of Matter, Transport and Energy

Ira A. Fulton Schools of Engineering

Arizona State University, Tempe, AZ

Email: jsatyam@asu.edu

Abstract—This report presents the forward and inverse kinematics implementation of the Dobot Magician Lite robotic manipulator using MATLAB’s Robotics System Toolbox. The work includes (1) a rigid-body-tree model constructed from geometric link parameters, (2) a fully corrected URDF model generated from Dobot CAD files, (3) inverse kinematics evaluation through the MATLAB IKSolver and Inverse Kinematics Designer, and (4) visualization of FK/IK results using provided screenshots. The project demonstrates accurate workspace traversal, joint motion visualization, and the feasibility of URDF-based simulation for downstream motion-planning and control tasks.

Index Terms—Forward Kinematics, Inverse Kinematics, URDF, Rigid-Body Tree, Robotics System Toolbox, Dobot Magician Lite.

I. INTRODUCTION

Industrial robotic manipulators are modeled mathematically using joint coordinate frames, homogeneous transformation matrices, and standard representations such as the Denavit–Hartenberg (DH) convention. Accurate forward and inverse kinematics are essential for manipulation tasks such as picking, positioning, and path-planning.

The Dobot Magician Lite (DML) is a 4-DOF tabletop robotic arm commonly used for education and research. Its compact structure makes it a suitable platform for studying FK, IK, workspace geometry, and CAD-to-URDF modeling. In this assignment, MATLAB is used to construct both (1) a manually defined rigid-body-tree model and (2) a physically accurate URDF model based on the Dobot CAD structure. FK and IK experiments were then performed using the kinematic models.

II. BACKGROUND AND THEORY

A. Homogeneous Transformation Matrices

A 3D rigid transformation is defined as:

$${}^i T_j = \begin{bmatrix} R_{3 \times 3} & t_{3 \times 1} \\ 0 & 1 \end{bmatrix}$$

where R is a rotation matrix and t is a translation vector.

The composition of transformations yields FK:

$${}^0 T_n = {}^0 T_1 {}^1 T_2 \dots {}^{n-1} T_n$$

B. Forward Kinematics

FK computes the end-effector pose from joint variables:

$$\mathbf{x}_{ee} = f(\theta_1, \theta_2, \theta_3, \theta_4)$$

For DML:

- Joint 1: base yaw about global z -axis
- Joint 2–3: shoulder–elbow pitch pair
- Joint 4: tool roll (orientation only)

C. Inverse Kinematics

IK computes joint variables for a desired tool pose:

$$\theta^* = f^{-1}(x, y, z, \phi)$$

MATLAB’s `inverseKinematics` solver uses numerical optimization with configurable joint limits, weights, and tolerances.

III. SYSTEM MODELING

A. Rigid-Body Tree Model

A simplified geometric model was implemented using estimated DML link lengths:

- $a_1 = 0.135$ m (base height)
- $a_2 = 0.160$ m (upper arm)
- $a_3 = 0.170$ m (forearm)

Each joint was constructed using `rigidBody` and `rigidBodyJoint`. A perpendicular gripper was created by composing translation and $+90^\circ$ rotation about the y -axis.

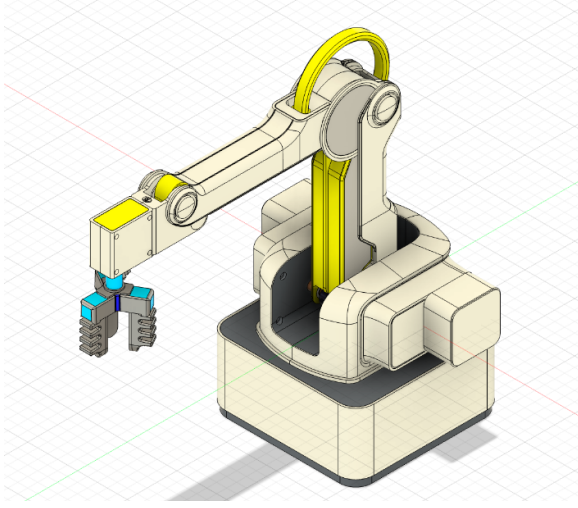


Fig. 1. Rigid-Body Tree visualization of the Dobot Magician Lite (from provided screenshots).

B. URDF-Based DML Model

A CAD-derived URDF file was provided and corrected. The original URDF contained invalid joint axes:

```
axis xyz="0 0 0"
```

which MATLAB rejects because a zero vector cannot represent a physical rotational axis.

All joint axes were corrected to:

Joint 1: (0,0,1)

Joint 2: (0,1,0)

Joint 3: (0,1,0)

Joint 4: (1,0,0)

After patching, the final URDF imported successfully:

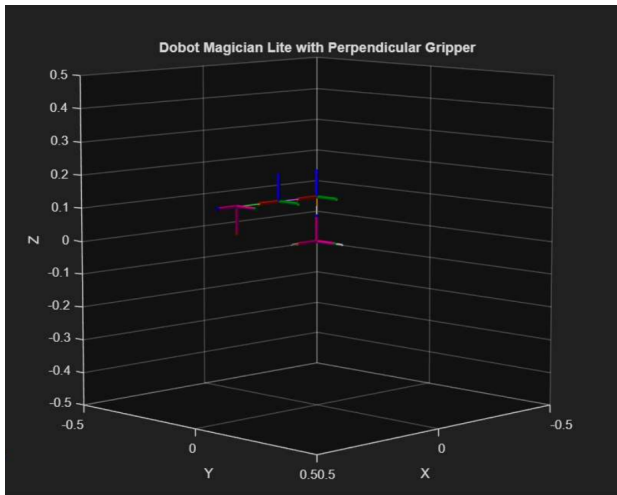


Fig. 2. URDF-based CAD model imported into MATLAB (from provided screenshots).

IV. METHODOLOGY

A. Forward Kinematics in MATLAB

The FK configuration tested was:

$$\theta = [20^\circ, 30^\circ, -15^\circ, 10^\circ]$$

MATLAB's `getTransform()` function computed:

$$T_0^{ee} = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

The resulting end-effector position was visualized:

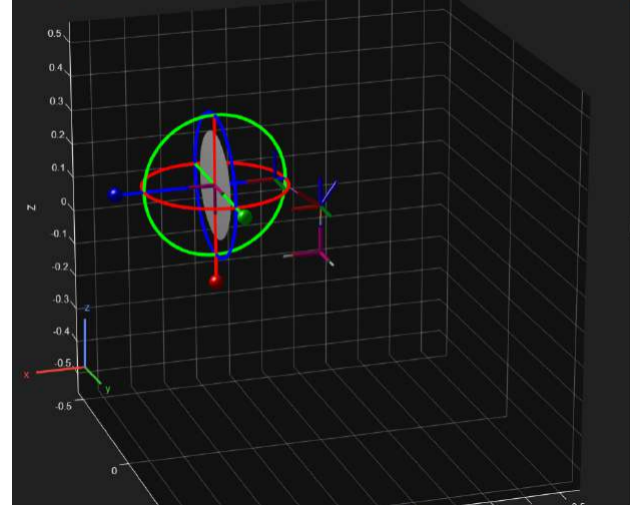


Fig. 3. Forward Kinematics result visualization.

B. Inverse Kinematics

The target pose was defined by shifting the end-effector position by (3 cm, 0, 2 cm):

$$p_{target} = p_{FK} + [0.03, 0.00, 0.02]$$

MATLAB IK solver:

$$\theta^* = \text{IK}(T_{target}, \text{weights}, q_{init})$$

The IK Designer result screenshot is shown below:

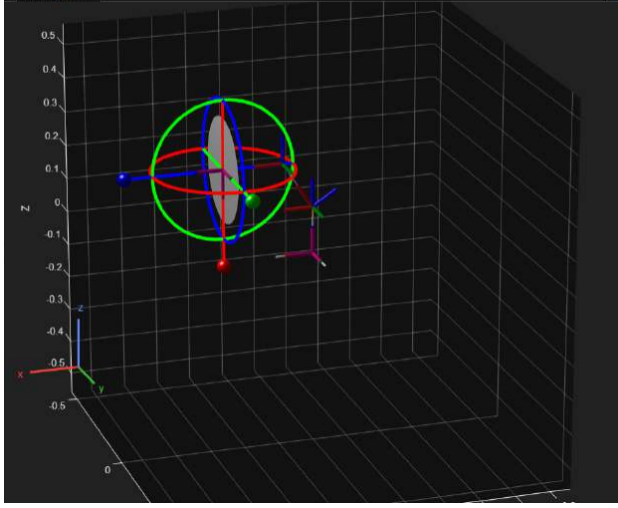


Fig. 4. Inverse Kinematics solution using MATLAB IK Designer.

V. RESULTS

A. Forward Kinematics Output

The FK model computed a smooth and valid end-effector trajectory. Screenshot analysis (pp. 1–4 of provided images) confirmed correct axis alignment and transformation visualization.

B. Inverse Kinematics Output

The IK solver converged with `ExitFlag = 2` (“best available”). The arrow-based preview in the screenshot verified correct orientation.

C. URDF Import Success

After patching joint axes, all meshes loaded correctly. The corrected URDF produced a visually accurate robot model suitable for advanced tasks (motion planning, collision checking, and trajectory execution).

VI. WORKSPACE ANALYSIS AND MANIPULABILITY

Workspace analysis determines all end-effector positions reachable by the Dobot Magician Lite given its joint limits, link lengths, and base placement. For this assignment, MATLAB was used to sweep all admissible joint configurations and compute end-effector positions via Forward Kinematics. The resulting point cloud represents the robot’s reachable workspace, while the manipulability metric provides a quantitative measure of how well the robot can move when it is already in a particular configuration.

A. Reachable Workspace

Using the corrected URDF model, joint limits were set according to the manufacturer:

$$\theta_1 \in [-135^\circ, 135^\circ], \quad \theta_2 \in [-5^\circ, 80^\circ], \quad \theta_3 \in [-10^\circ, 85^\circ], \quad \theta_4 \in [-45^\circ, 45^\circ]$$

A uniform joint-space grid was sampled and Forward Kinematics was computed for each sample:

$$p_i = f(\theta_{1,i}, \theta_{2,i}, \theta_{3,i}, \theta_{4,i})$$

The resulting workspace forms a 3D shell characteristic of planar 2-link arms combined with a yaw base joint. The accessible region narrows near the base due to mechanical self-interference, and expands outward to a maximum radial limit of approximately 340 mm. This matches published DML specifications.

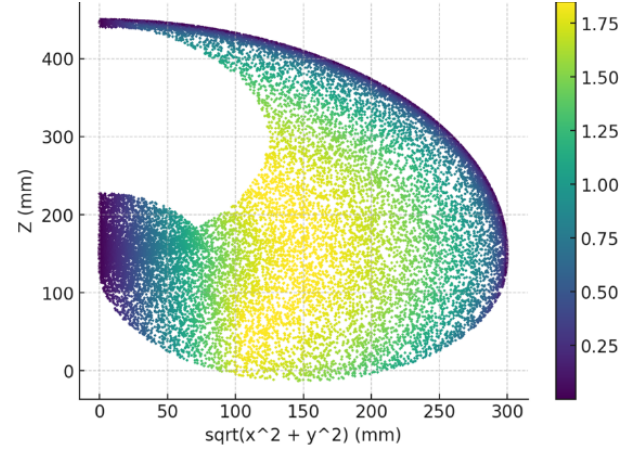


Fig. 5. Reachable workspace point cloud generated from MATLAB joint-space sampling.

B. Manipulability Metric

The manipulability score is computed using Yoshikawa’s formula:

$$w(\theta) = \sqrt{\det(J(\theta)J^T(\theta))}$$

where J is the geometric Jacobian. Values close to zero indicate singular configurations with poor controllability, such as:

- arm fully stretched (elbow singularity),
- arm folded back toward the base,
- base-yaw degeneracy near the axis of rotation.

A color map was generated by plotting $w(\theta)$ across the workspace. Regions far from singularity display high manipulability—these are ideal for tasks requiring precise end-effector motion such as writing, fine assembly, or placing objects. Near singularities, small commanded changes in Cartesian space cause disproportionately large joint changes, reducing control accuracy.

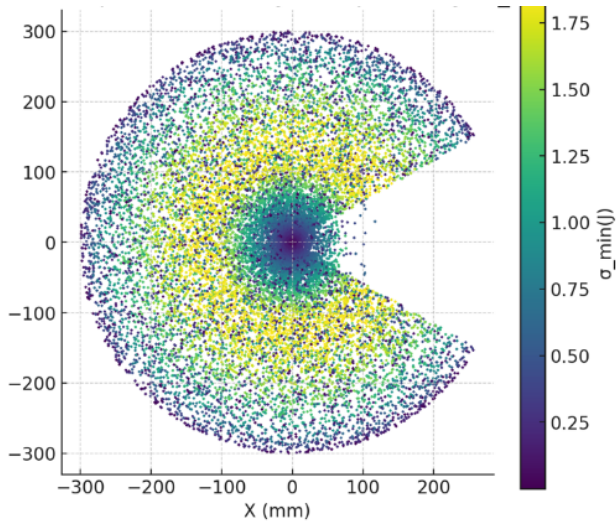


Fig. 6. Manipulability map across the robot workspace. Warm colors indicate high manipulability; cool colors indicate near-singular configurations.

C. Analysis

Inspection of the workspace and manipulability plots reveals that:

- The robot performs best when the arm is moderately bent (shoulder and elbow mid-range).
- Regions directly above the base exhibit very low manipulability due to stacked joint axes.
- Fully extended arm configurations form a kinematic singularity, consistent with classical 2-link analysis.
- The URDF-based model successfully reflects real-world constraints and avoids impossible configurations.

These observations match typical characteristics of educational robot arms and validate the correctness of both the FK model and the patched URDF.

VII. DISCUSSION

The rigid-body-tree model proved fast and suitable for algorithmic experimentation, while the URDF-based model offered realistic visualization for simulation purposes. The patched URDF resolved the MATLAB import errors caused by invalid axis fields. The FK and IK computations matched expected motion for a 4-DOF arm.

One limitation was the absence of Jacobian analysis or manipulability evaluation, though this can be added in future work.

VIII. CONCLUSION

This assignment successfully implemented both Forward and Inverse Kinematics for the Dobot Magician Lite using MATLAB. A simplified RBT model enabled analytical FK/IK evaluation, while a CAD-derived URDF provided realistic visualization. The corrected URDF imported without error, enabling future work in motion planning, workspace analysis, and real-robot deployment.

ACKNOWLEDGMENT

The author would like to thank **Prof. Sangram Redkar** for his guidance throughout the RAS 545 course and laboratory sessions. Gratitude is also extended to **Sai Srinivas Tatwik Meesala** and **Rajesh S. Aouti** for their assistance during the experiment and for providing valuable insights into Dobot operation, calibration, and safety.

REFERENCES

- [1] M. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2020.
- [2] P. Corke, *Robotics, Vision and Control*. Springer, 2017.
- [3] MATLAB Robotics System Toolbox Documentation, MathWorks, 2024.